UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

SEDE DE SANTO DOMINGO

PERIODO:

Marzo 2026 – agosto 2026 (202650)

ASIGNATURA:

Metodología de Desarrollo Software

TEMA:

Investigación sobre Requerimientos Funcionales y No Funcionales

ESTUDIANTE:

Jeffry Vera

NRC:

29531

DOCENTE:

Ing. Kevin Chuquitarco

SANTO DOMINGO - ECUADOR

Índice

Introducción 3

Objetivos 4

Objetivo general 4

Objetivos específicos 4

Desarrollo 5

Requerimientos funcionales 5

Requerimientos no funcionales 6

Buenas prácticas en la especificación de requerimientos 7

Formas de especificar requerimientos 8

Especificación en lenguaje natural 8

Historias de usuario 9

Diagrama de clases 11

Conclusiones 13

Recomendaciones 14

Bibliografía 15

Investigación sobre Requerimientos Funcionales y No Funcionales

Introducción

Antes de programar algo hay que saber qué se va a programar. Suena obvio pero buena parte de los proyectos de software fracasan no por errores de código sino porque nadie definió con claridad qué debía hacer el sistema ni bajo qué condiciones tenía que funcionar. Ahí entra el desarrollo de software: la disciplina encargada de capturar y validar esas necesidades antes de que se conviertan en diseño y después en código (Pichler, 2010).

Por convención estos requerimientos se dividen en dos grupos. Los funcionales describen qué acciones concretas debe ejecutar el sistema: registrar un usuario, calcular un total, generar un reporte. Los no funcionales se ocupan de cómo debe comportarse ese sistema en términos de rendimiento, seguridad o facilidad de uso (Sommerville, 2015). En la teoría la línea entre ambos parece nítida. En la práctica no tanto. Es común que un mismo requerimiento redactado por un usuario mezcle sin darse cuenta una funcionalidad con una restricción de calidad y separar eso bien exige criterio de quien analiza el proyecto (Cohn, 2010).

La forma de documentar estos requerimientos también cambió con los años. Las especificaciones extensas en lenguaje natural tan típicas de los enfoques tradicionales fueron perdiendo terreno frente a técnicas más ágiles como las historias de usuario. Mike Cohn las popularizó en User Stories Applied (2004) y suelen evaluarse con el criterio INVEST que Bill Wake propuso en 2003. Cuando el sistema involucra varias entidades relacionadas entre sí el diagrama de clases de UML sigue siendo la herramienta de referencia para representar esa estructura de forma gráfica (Sommerville, 2011).

Este trabajo recorre esos tres elementos requerimientos funcionales, no funcionales y las formas de especificarlos con el objetivo de entender no solo qué son, sino cómo se aplican correctamente en un proyecto real de desarrollo de software.

Objetivos

Objetivo general

Analizar los requerimientos funcionales y no funcionales como componentes fundamentales del desarrollo de software, examinando sus características, buenas prácticas y formas de especificación aplicadas en el desarrollo de software.

Objetivos específicos

- Definir los requerimientos funcionales y no funcionales, identificando sus características distintivas y su clasificación según la literatura especializada.
- Describir las buenas prácticas reconocidas para la correcta especificación de requerimientos de software, conforme a estándares como el IEEE 830.
- Explicar las principales formas de especificar requerimientos: el lenguaje natural estructurado, las historias de usuario bajo el criterio INVEST y el diagrama de clases UML.
- Comparar las ventajas y limitaciones de cada forma de especificación, determinando los contextos en los que su aplicación resulta más adecuada.

Desarrollo

Requerimientos funcionales

Un requerimiento funcional define qué debe hacer el sistema ante una entrada determinada: qué procesa, cómo se comporta y qué resultado entrega (Wiegers y Beatty, 2020). Aquí entran los cálculos las validaciones la manipulación de datos y cualquier proceso de negocio que el software tenga que ejecutar su contenido varía muchisimo según el dominio. Los requerimientos funcionales de un sistema bancario no se parecen en nada a los de un videojuego aunque ambos se documenten con la misma lógica.

Sommerville (2005) lo resume bien: estos requerimientos describen las transformaciones que el sistema aplica sobre sus entradas para producir salidas. Lo importante acá es no confundir el qué con el cómo. Un requerimiento funcional bien redactado dice qué debe lograr el sistema y no cómo se va a programar esa lógica porque eso le corresponde a la etapa de diseño. Una especificación completa además deja todos los servicios solicitados por el usuario claramente definidos sin contradicciones entre uno y otro (UV, s.f.).

El estándar del IEEE marca tres características que todo requerimiento funcional debería cumplir. Corrección: que refleje una necesidad real del sistema además Ausencia de ambigüedad: que admita una sola interpretación posible Y completitud: tener respuestas definidas para cualquier entrada válida o inválida (UNIPACIFICO, s.f.). En la práctica estos requerimientos suelen plasmarse en casos de uso que el analista construye después de identificar cómo interactúa el usuario con el sistema.

Requerimientos no funcionales

Si los requerimientos funcionales responden a “qué hace el sistema” los no funcionales responden a “qué tan bien lo hace”. No describen una función puntual sino propiedades que atraviesan todo el sistema: fiabilidad, rendimiento, usabilidad, seguridad, tiempo de respuesta (Sommerville, 2005). Son las condiciones bajo las cuales el sistema tiene que operar para que valga la pena usarlo (pmoinformatica, 2015).

Y acá hay algo que conviene tener presente: estos requerimientos suelen pesar más que cualquier funcionalidad individual. Un sistema puede tener todas sus funciones implementadas correctamente y aun así ser inútil si tarda demasiado en responder o si no protege la información del usuario (UV, s.f.). La Tabla 1 organiza estos requerimientos en tres grupos siguiendo la clasificación que propone Sommerville.

Tabla 1

Clasificación de los requerimientos no funcionales según Sommerville

Categoría	Descripción
Requerimientos de producto	Especifican el comportamiento del producto, como la usabilidad esfuerzo requerido para aprender y usar el sistema, la eficiencia, la portabilidad y la confiabilidad.
Requerimientos organizacionales	Se derivan de políticas y procedimientos de la organización del cliente y del desarrollador; incluyen estándares de codificación, metodologías y herramientas de soporte al desarrollo.
Requerimientos externos	Se originan en factores externos al sistema y a su proceso de desarrollo, como requerimientos legales, éticos, de interoperabilidad o normativos.

Nota. Elaboración propia basada en la clasificación de requerimientos no funcionales propuesta por Sommerville, según lo sistematizado por pmoinformatica (2015).

Vale la pena aclarar que esta separación entre funcional y no funcional es más limpia en la teoría que en la práctica. Un usuario que redacta un requerimiento en lenguaje natural rara vez separa ambos tipos por su cuenta. Suele mezclarlos en la misma frase y es tarea del analista detectar dónde termina la funcionalidad y dónde empieza la restricción de calidad (UV, s.f.).

Buenas prácticas en la especificación de requerimientos

Da igual qué técnica de especificación se use los requerimientos se documentaron mal desde el inicio el problema se arrastra a lo largo de todo el proyecto. La literatura coincide en algunas prácticas básicas que ayudan a evitarlo. La primera es bastante intuitiva: cada requerimiento debe ser claro, conciso, completo y verificable para que tanto el equipo de desarrollo como los interesados entiendan exactamente lo mismo al leerlo (Scribd, 2024).

También importa priorizar. No todos los requerimientos valen lo mismo para el negocio así que conviene identificar cuáles generan mayor valor y validarlos directamente con los interesados antes de empezar a programar. De lo contrario se corre el riesgo de construir algo que el cliente nunca pidió (Scribd, 2024). Otra práctica que rinde frutos más adelante es definir junto con cada requerimiento cómo se va a probar que efectivamente se cumplió. Esto facilita muchísimo el trabajo del equipo de pruebas (LACCEI, 2010).

Y por último algo que muchos pasan por alto: la trazabilidad. El estándar IEEE 830 para Especificaciones de Requisitos de Software exige que cada requerimiento pueda rastrearse hasta su origen una necesidad del usuario un estándar un contrato y hasta los elementos de diseño y código que finalmente lo implementan .Sin esa trazabilidad es difícil justificar por qué existe cada parte del sistema.

Formas de especificar requerimientos

No hay una única manera correcta de documentar requerimientos. Existen distintas notaciones que van desde el lenguaje natural flexible pero también el más propenso a malentendidos hasta especificaciones matemáticas formales pasando por notaciones gráficas y lenguajes más estructurados (UV, s.f.). De todas ellas tres resultan particularmente relevantes hoy en día: el lenguaje natural estructurado, las historias de usuario y el diagrama de clases.

Especificación en lenguaje natural

Es históricamente la forma más usada para documentar requerimientos por una razón simple: no exige que el usuario sepa nada de notaciones técnicas. Sommerville lo plantea como declaraciones en texto o en diagramas de los servicios que el sistema debe ofrecer y de las restricciones bajo las cuales tiene que operar (como se citó en Revista Cubana de Informática Médica, 2020).

El problema es que el lenguaje natural sin ningún tipo de control es ambiguo casi por definición. Una misma frase puede interpretarse de dos formas distintas. Varios requerimientos pueden terminar mezclados en un solo párrafo. Y representar relaciones complejas entre requisitos se vuelve casi imposible solo con texto corrido (UV, s.f.). Por eso surgió el lenguaje estructurado: una versión más disciplinada que toma prestadas algunas construcciones de la programación condicionales, listas, plantillas fijas para reducir esa ambigüedad sin perder la posibilidad de que cualquiera lo entienda (UV, s.f.).

En la práctica esto se traduce en un par de hábitos sencillos. Mantener siempre la misma estructura gramatical para requerimientos del mismo tipo. Evitar la jerga técnica que confunde a los interesados que no son del área. Y nunca meter dos requerimientos distintos en una sola oración porque eso complica tanto la trazabilidad como las pruebas posteriores.

Historias de usuario

Una historia de usuario es básicamente un requerimiento reducido a su mínima expresión: una o dos frases escritas con el vocabulario del usuario y no del programador. La idea nació dentro de la Programación Extrema (XP) hacia 1999 pero fue Mike Cohn quien la terminó de consolidar como estándar en User Stories Applied: For Agile Software Development (2004), el libro que fijó el patrón de redacción que se sigue usando hoy (Netmind, 2026).

Ese patrón suele verse así: “Como [rol], quiero [funcionalidad], para [beneficio]”. El rol identifica a quién le interesa la historia. La funcionalidad describe qué quiere. Y el beneficio explica para qué lo quiere, qué valor de negocio hay detrás (DosIdeas, s.f.). Lo importante es no tratarla como un contrato cerrado. Una historia de usuario es apenas el punto de partida de una conversación entre el equipo y el dueño del producto, donde recién ahí se acuerdan los detalles finos de implementación (Scrum Manager, 2026).

Ahora bien no toda frase corta cuenta como una buena historia de usuario. Bill Wake propuso en 2003 el acrónimo INVEST que Cohn terminó de popularizar para evaluar si una historia está realmente lista para entrar al backlog o si necesita reescribirse (Scrum Manager, 2026). La Tabla 2 resume sus seis criterios.

Tabla 2

Características del criterio INVEST para historias de usuario

Letra	Característica	Descripción
I	Independiente	La historia puede planificarse e implementarse en cualquier orden, sin depender de otras historias.
N	Negociable	No es un contrato cerrado; sus detalles se acuerdan mediante conversación entre el equipo y el cliente.
V	Valiosa	Debe aportar un valor tangible y comprensible al usuario o al negocio.
E	Estimable	El equipo debe poder valorar, con suficiente precisión, el esfuerzo necesario para implementarla.
S	Pequeña (Small)	Debe completarse en un periodo corto, idealmente días; si es muy grande, se trata de una épica que debe dividirse.
T	Comprobable (Testable)	Debe contar con criterios de aceptación claros que permitan verificar objetivamente que fue completada.

Nota. Adaptado de "INVEST," por Scrum Manager BoK, 2026.

Además de INVEST hay otro principio que conviene tener presente: las 3C. Card porque la historia debe caber en una tarjeta o ficha breve. Conversation porque su significado real se construye en el diálogo continuo entre quien pide algo y quien lo va a implementar y no en el texto inicial. Confirmation son los criterios de aceptación que finalmente determinan si la historia quedó bien hecha (Caroli, 2024).

Un detalle que se presta a confusión: las historias de usuario no aparecen formalmente en la Guía de Scrum aunque su popularidad creció de la mano de este marco de trabajo. Por eso mucha gente las usa como sinónimo de "requerimiento" dentro del backlog cuando en realidad son apenas una de las formas posibles de expresarlo (Netmind, 2026). También se pueden escribir historias de usuario sobre requerimientos no funcionales tiempos de respuesta, seguridad y en ese caso quien las redacta suele ser un usuario técnico o de soporte y no el cliente final (pmoinformatica, 2014).

Diagrama de clases

Cuando un sistema involucra varias entidades relacionadas entre sí ni el lenguaje natural ni las historias de usuario alcanzan para representar esa estructura con claridad. Ahí entra el diagrama de clases: un tipo de diagrama estático de UML que muestra las clases del sistema, sus atributos, sus operaciones y cómo se relacionan entre sí (Diagramasuml, 2025). Con el tiempo se ha vuelto una de las herramientas más usadas en el diseño orientado a objetos porque permite que un equipo entero desarrolladores, arquitectos, analistas visualice la estructura del sistema antes de escribir una sola línea de código.

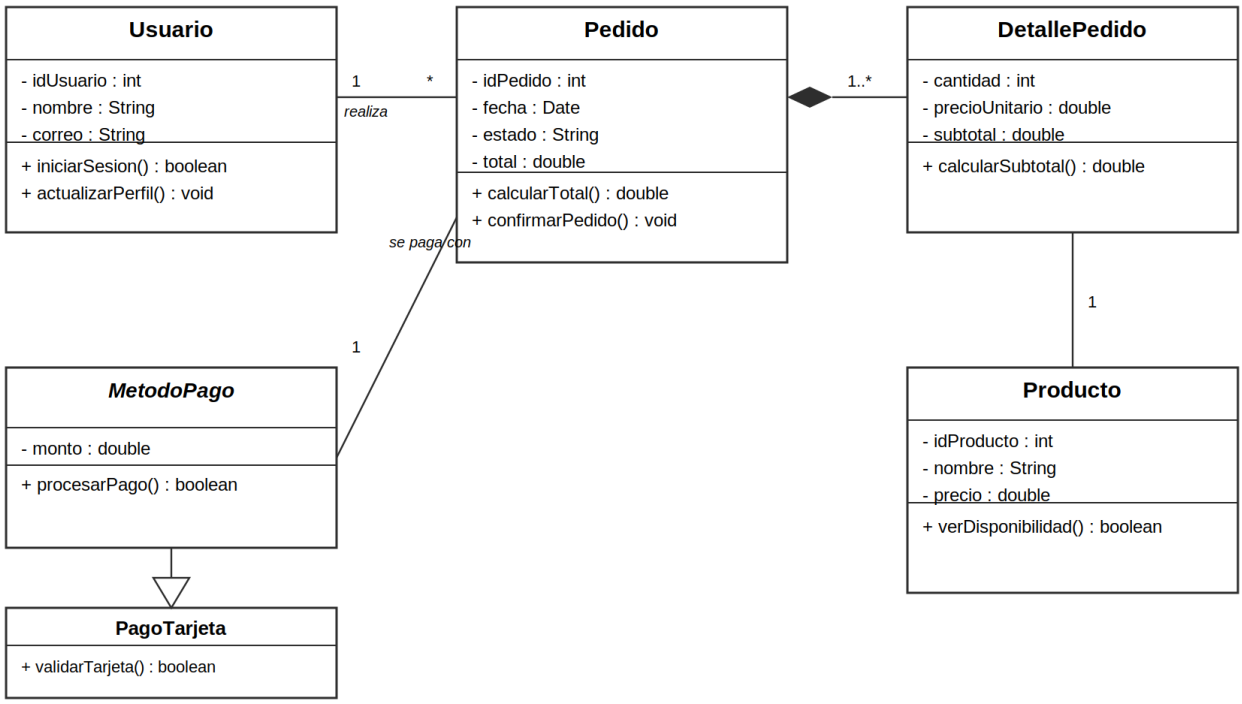
Cada clase se dibuja como un rectángulo dividido en tres partes. Arriba va el nombre. En el medio los atributos con su tipo de dato. Y abajo las operaciones con su firma correspondiente (Diagrams-AI, 2026). Para indicar quién puede acceder a cada atributo u operación UML usa símbolos de visibilidad bastante intuitivos: el más (+) para lo público, el menos (−) para lo privado y la almohadilla (#) para lo protegido (Lucidchart, 2026).

Las clases casi nunca van solas. Se conectan mediante relaciones que UML representa con notaciones distintas según el tipo de vínculo. La asociación la más genérica se dibuja con una línea continua y suele llevar indicadores de multiplicidad que dicen cuántas instancias de una clase pueden relacionarse con otra. La herencia en cambio representa una relación “es-un” entre una clase hija y su clase padre y se dibuja con una línea que termina en un triángulo hueco apuntando hacia la superclase. Así la clase hija hereda todos sus atributos y operaciones (Canva, 2026).

Para que estos conceptos no se queden solo en la teoría la Figura 1 muestra un diagrama de clases sencillo: un sistema de gestión de pedidos con una asociación entre Usuario y Pedido, una composición entre Pedido y DetallePedido y una relación de herencia entre la clase abstracta MetodoPago y su clase hija PagoTarjeta.

Figura 1

Diagrama de clases UML de un sistema genérico de gestión de pedido



Nota. Diagrama de clases UML de un sistema genérico de gestión de pedidos. Elaboración propia (2026), basada en la notación descrita por Diagramasuml (2025).

La multiplicidad entre Usuario y Pedido indica que un mismo usuario puede tener múltiples pedidos asociados a lo largo del tiempo. La relación entre Pedido y DetallePedido se marca con un rombo relleno en el extremo de Pedido. Esa es la notación de composición: una forma de agregación bastante estricta donde los detalles de un pedido no tienen sentido sin el pedido al que pertenecen (Miro, 2025). Y la flecha de triángulo hueco entre PagoTarjeta y MetodoPago esta última en cursiva como corresponde a una clase abstract muestra cómo distintos métodos de pago heredarían el comportamiento común definido en la superclase (Diagrams-AI, 2026).

Más allá del ejemplo el diagrama de clases gana protagonismo justamente cuando el lenguaje natural y las historias de usuario empiezan a quedarse cortos. Cuando hay muchas entidades de negocio relacionadas entre sí una imagen estructurada comunica eso mucho mejor que cualquier párrafo, por bien redactado que esté (UV, s.f.).

Conclusiones

Separar requerimientos funcionales de no funcionales no es un ejercicio teórico sin aplicación real de esa distinción depende si un sistema termina sirviendo o no. Los primeros definen qué hace el sistema los segundos definen si eso vale la pena en la práctica. Esto se nota especialmente en sistemas con procesos sensibles al tiempo o a la seguridad un módulo puede cumplir toda su lógica funcional y aun así fracasar si el proceso tarda demasiado en responder o si no protege correctamente la información del usuario. Esto confirma algo que va más allá de la clasificación teórica los requerimientos no funcionales deberían priorizarse desde el arranque del proyecto, y no tratarse como una revisión de calidad recién al final del desarrollo.

Sobre las buenas prácticas de especificación, la conclusión es igual de directa: claridad, verificabilidad y trazabilidad no son formalismos para que un documento se vea prolijo, sino la condición real para que un requerimiento oriente correctamente el diseño y las pruebas posteriores. Cuando un requerimiento queda ambiguo, el problema no desaparece con el avance del proyecto simplemente se traslada a una etapa posterior, donde corregirlo cuesta más tiempo, más recursos y en varios casos obliga a rehacer trabajo que el equipo ya había entregado.

En cuanto a las tres formas de especificación revisadas, ninguna resuelve todos los escenarios por sí sola, y esa es justamente la reflexión central de este trabajo: la elección de técnica

no debería decidirse una sola vez al inicio del proyecto, sino ajustarse según la complejidad que el sistema va mostrando durante el análisis. El lenguaje natural estructurado es el más accesible para usuarios sin formación técnica, pero exige disciplina constante para no caer en ambigüedad. Las historias de usuario evaluadas con INVEST rinden mejor en proyectos ágiles, donde la conversación constante con el cliente reemplaza a la documentación extensa, aunque pierden valor si el equipo las trata como un checklist en vez de un punto de partida para negociar. Y el diagrama de clases se vuelve indispensable justo donde el texto, por bien redactado que esté deja de alcanzar: sistemas con varias entidades relacionadas entre sí, donde una estructura visual comunica en un vistazo lo que un párrafo tardaría varias líneas en explicar. Combinar estas tres técnicas, más que elegir una sola de forma definitiva, parece ser la estrategia más razonable para cualquier proyecto de mediana o alta complejidad.

Recomendaciones

- Definir desde el arranque del proyecto una plantilla común para redactar requerimientos en lenguaje natural, así todo el equipo usa la misma estructura y se reduce la ambigüedad entre documentos distintos.
- Usar INVEST como guía de conversación, no como checklist mecánico, al redactar historias de usuario; lo que importa es entender bien la historia, no marcar las seis letras del acrónimo.
- Armar el diagrama de clases en las primeras etapas del análisis cuando el sistema tenga varias entidades relacionadas, y revisarlo junto al equipo técnico antes de empezar a codificar, para detectar errores estructurales mientras todavía son baratos de corregir.

- Acompañar cada requerimiento, sin importar cómo se haya especificado, con un criterio de aceptación verificable, de modo que el equipo de pruebas tenga una base objetiva para validar si realmente se cumplió.

Bibliografía

Caroli, P. (2024). Historias de usuario y creación de productos exitosos. Caroli.org. https://caroli.org/es/historias-de-usuario/

Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley.

Diagramas-AI. (2026). La guía completa sobre los diagramas de clases UML: conceptos, notación y mejores prácticas. https://www.diagrams-ai.com/es/the-complete-guide-to-uml-class-diagrams-concepts-notation-and-best-practices-2/

Diagramasuml. (2025). Diagrama de clases. Teoría y ejemplos. https://diagramasuml.com/diagrama-de-clases/

DosIdeas. (s.f.). El modelo INVEST para crear historias de usuario efectivas. https://dosideas.com/noticias/metodologias/980-el-modelo-invest-para-crear-historias-de-usuario-efectivas

Lucidchart. (2026). Tutorial de diagramas de clases UML. https://www.lucidchart.com/pages/es/tutorial-de-diagrama-de-clases-uml

Miro. (2025). Diagrama de clases: qué es, cómo hacerlo y ejemplos. https://miro.com/es/diagrama/que-es-diagrama-clases-uml/

Netmind. (2026). Historias de usuario: qué son, reglas y consejos para su aplicación. https://netmind.net/es/historias-de-usuario-reglas/

pmoinformatica. (2015). Requerimientos no funcionales: una clasificación. https://www.pmoinformatica.com/2015/04/requerimientos-no-funcionales-una.html

Redalyc. (s.f.). Requerimientos de software: un enfoque práctico para su gestión. InterSedes: Revista de las Sedes Regionales. https://www.redalyc.org/pdf/666/66612870011.pdf

Scribd. (2024). Requerimientos funcionales y no funcionales. https://es.scribd.com/document/844931731/Requerimientos-Funcionales-y-No-Funcionales-en-La-Ingenieria-de-Requerimientos

Scrum Manager. (2026). INVEST. Scrum Manager BoK. https://www.scrummanager.com/bok/index.php/INVEST